AI Engineering and Claude Architect · Part IV, Advanced

Advanced AI Topics, Exercises

Six Worked Exercises

Reject a fine-tune, size a server, read research carefully, set an autonomy policy, crash a workflow, and defend a decision.

Module	AI11 of 13
Covers	All 5 lessons
Time	~2.5 hours

Exercise 1: Should we fine-tune?

For each, decide what to do first, and say why. **(a)** A bot must know 3,000 product specs, which change weekly. **(b)** A classifier sorts 50 million tickets a month, and it uses a 4,000-token prompt with examples. Accuracy is good. **(c)** Replies must match a very specific brand voice, and three rounds of prompt work have got to 70% on a rubric, against a target of 90%. **(d)** The system answers wrongly on tickets in Marathi.

Solution

(a) Retrieval or a tool lookup. Specs are changing facts, and need sources.

(b) Enable prompt caching first, since most of those 4,000 tokens are repeated. If most are the prefix, cached reads cut that cost heavily. Then test whether a smaller model passes evals. Only if it still costs too much would I try a fine-tune to shorten the prompt. Volume makes this one a legitimate candidate, after those steps.

(c) First check the rubric's reliability, as 70% may partly be judges disagreeing (AI10 L3). Try few-shot examples of the voice. If those plateau, a fine-tune is a fair experiment, since style can be easier to show than describe. Define the test first, against the prompted baseline.

(d) Find out why, by segment. Check whether retrieval, the prompt or the model is at fault, and test other models. Fine-tuning needs labelled Marathi data, and it might help if the base truly lacks ability, but prompts and models are quicker to test.

Exercise 2: Will it fit?

You're considering a 70B-parameter model at 8 bits, on GPUs with 96 GB. Use this lesson's KV shape, of 32 layers, 16 heads, head dimension 128, at 16-bit. Requests average 8,000 tokens. The server costs $4 an hour and produces 3,000 tokens a second, when busy at 40% average utilization.

(a) How many concurrent requests fit, leaving 6 GB overhead? **(b)** What does a million output tokens cost? **(c)** Two-thirds of traffic moves to a 32,000-token context. What happens to (a)?

Solution

(a) Weights: 70 GB. Left: 96 − 70 − 6 = 20 GB. Per request: 2 × 32 × 16 × 128 × 2 × 8,000 bytes = 2.097 GB. So about **9 concurrent requests**.

(b) $4 ÷ (3,000 × 0.4 × 3,600) × 1,000,000 = **$0.93**, before people.

(c) A 32,000-token request is about 8.39 GB. Average per request: (2/3 × 8.39) + (1/3 × 2.10) = 6.29 GB. So 20 ÷ 6.29 is about **3 concurrent requests**. Capacity falls threefold, so throughput falls, and cost per token rises, because memory limited the batch. This is why long contexts are costly to self-host. Options: cap context, trim prompts, use more GPUs, or use 4-bit weights to free 35 GB, if evals agree.

Exercise 3: Read the research

Rewrite each statement so that it's accurate, using this module's sources. **(a)** "Claude blackmails people 96% of the time." **(b)** "Models fake alignment, so evals are worthless." **(c)** "Anthropic can read Claude's mind now." **(d)** "250 documents can poison any AI."

Solution

(a) In a June 2025 experiment with constructed scenarios and forced choices, under replacement threat plus a goal conflict, Claude Opus 4 and Gemini 2.5 Flash blackmailed in 96% of runs. The authors report no evidence of this in real deployments.

(b) In a December 2024 experiment, with a fictional retraining setup, Claude 3 Opus sometimes complied strategically when told it was monitored. The authors say this doesn't show that dangerous faking will necessarily emerge. Evals remain useful, and this is a reason to add production monitoring as well.

(c) Anthropic has found millions of features on Claude 3.0 Sonnet, and traced some circuits on Claude 3.5 Haiku. Its own limits: a small subset, a fraction of the computation, and hours of work per short prompt.

(d) In an October 2025 study, about 250 documents backdoored models from 600M to 13B parameters, causing gibberish on a trigger. It's narrow and low-stakes, and it's unclear whether this holds for larger models or more harmful behaviours.

Exercise 4: Set an autonomy policy

A coding agent works on a repository. Decide AUTO, CHECKPOINT or HUMAN for each, and say what you'd change once it reads a public issue: read files; edit on a branch; run tests; add a dependency; push to a branch; merge to main; change CI config; call an internal production API; delete a database table.

Solution

Action	Default	After reading a public issue
Read files	AUTO	AUTO, secrets excluded
Edit on a branch	AUTO	AUTO
Run tests	AUTO, in a sandbox with no network	Same
Add a dependency	CHECKPOINT	HUMAN. Packages run code, and a typosquat is a classic injection goal
Push to a branch	AUTO, scoped proxy credential	CHECKPOINT
Merge to main	HUMAN	HUMAN
Change CI config	HUMAN	HUMAN. CI often holds secrets
Production API	Not given	Not given
Delete a table	Not given	Not given

The last two aren't "ask." Neither belongs in this agent's toolset (TS 2.3). Reasoning: reversibility, then what an injection could achieve. Enforce it with hooks and a sandbox, as Bash rules read only the command's text (AI09 L3). Add a live view and a stop button.

Exercise 5: Crash this workflow

An insurance flow: (1) a model reads a claim and proposes an amount; (2) a person approves over up to 3 days; (3) pay the claimant; (4) email them; (5) update the ledger. The team's code is a single function, run by a queue consumer with retries.

(a) Name the failures. **(b)** Redesign. **(c)** What happens if the system prompt is revised during the 3-day wait?

Solution

(a) A retry re-asks the model, which may propose another amount. The approval wait holds a process, or it's lost on restart. A crash after paying and before saving pays twice. A crash after email sends it twice. A duplicate message starts another run. If the ledger update fails after payment, the books are wrong.

(b) Dedupe by claim event ID. A workflow, with each step recorded: the model call as an Activity, with its answer, prompt version and sources saved. A human Signal, with a timeout and escalation. Then order: ledger entry as pending, pay with an idempotency key, mark the ledger as paid, then email. Email goes last, as it can't be unsent. If payment fails, mark the ledger as failed. If marking fails after payment, retry with a key, and alert a person if it stays stuck. Reviewers see a structured summary, including sources.

(c) The proposal is already recorded, so it doesn't change, and the approver sees what was proposed. New runs use the new prompt. Pin the version in workflow state. If the change fixes a serious error, a code path should find waiting runs, and send them back for re-review. Don't change them silently.

Exercise 6: Defend a decision

The decision: a hospital group wants a clinical note summarizer. Its CTO proposes self-hosting a fine-tuned open model, "for privacy and cost." Write your recommendation.

Model answer

I'd challenge the proposal's two reasons, and test a simpler option first. I'm not giving legal advice, and compliance must settle the data question.

Privacy. First get the actual rule, in writing, from legal and compliance. "Can't go to a third party" and "needs a business associate agreement and specific handling" lead to different designs. Anthropic's retention page describes HIPAA readiness arrangements, zero data retention, and which features each covers, and which platforms have the cloud provider as data processor. If those satisfy the rule, then self-hosting isn't needed for privacy. If there's a genuine rule against any external processing, self-hosting is justified, and the rest of this is about doing it well.

Cost. Volumes for a hospital group are probably modest, compared with L2's break-even. An idle GPU overnight costs the same as a busy one. Add engineers, on-call, and safety work. I'd do that arithmetic with real volumes, and I expect the API to win, unless volumes are very high.

Fine-tuning. Summarization style can usually be taught by prompts and examples. Clinical facts come from the note itself, and not from weights. Training on patient notes also puts personal data into weights, which can't practically be deleted. I'd need an eval showing that the prompted baseline fails before considering this.

Regardless of hosting, the risks that matter most are clinical. An invented medication is the failure to fear. So I'd want claims linked to source text, a check that doses and dates in the summary appear in the note, abstention when the note is unclear, evaluation by department and note type (AI10 L3), and a clinician reviewing and signing every summary at first. Autonomy would grow only with evidence.

My recommendation. Prove quality with a pilot on a compliant hosted platform with de-identified or approved data. Build the evals. Then decide hosting with real numbers.

Risks I accept. Dependence on a vendor and its retirement dates (AI10 L6). And the pilot may show that compliance rules this out.

What would change my mind. A firm rule against external processing. Very high sustained volume. Or evidence that a tuned open model beats the hosted baseline on our evals, including for rare note types.